Research-Grade Training Process Documentation

EmpatheticDialogues Emotion Classification using TF-IDF and Linear SVM

Google Colab Machine Learning Pipeline Documentation

Prepared for GitHub repository documentation and academic reporting

Source pipeline: empathetic-dialogues-tfidf-svm-emotion-classifier.pdf, 19-page Colab export, uploaded by the user.

Executive Snapshot

Item	Summary
Dataset	EmpatheticDialogues Kaggle CSV: emotion-emotion_69k.csv
Task	8-class text emotion classification mapped to Plutchik categories
Best Model	LinearSVC, C=0.3, class_weight=balanced
Split Strategy	Group-safe train/validation/test split by normalized situation group
Train / Validation / Test	44,651 / 9,650 / 9,604 samples
Feature Space	Word TF-IDF + character TF-IDF, 315,394 combined dimensions
Final Test Accuracy	0.7000
Final Macro F1 / Weighted F1	0.6786 / 0.7000

Abstract

This document describes the full training process for a text-based emotion classification pipeline implemented in Google Colab. The pipeline downloads the EmpatheticDialogues dataset from Kaggle, maps original emotional annotations into eight Plutchik-inspired categories, cleans the dataset, prevents situation-level data leakage, extracts word-level and character-level TF-IDF features, trains multiple Linear Support Vector Machine configurations, and evaluates the best model using accuracy, precision, recall, F1-score, class support, and confusion matrix analysis. The final selected model is a balanced LinearSVC classifier with C=0.3. It achieved 0.7000 test accuracy, 0.6786 macro F1, and 0.7000 weighted F1 on the held-out test set.

Keywords

Emotion classification; EmpatheticDialogues; Plutchik emotions; TF-IDF; Linear SVM; text classification; machine learning; Google Colab; confusion matrix; macro F1.

1. Project Overview

The training pipeline is designed as a classical machine learning baseline for emotion classification. Rather than using transformer-based contextual embeddings, it represents dialogue text using TF-IDF n-gram features and trains a LinearSVC classifier. This makes the system lightweight, reproducible, and suitable for environments where GPU-intensive transformer training is not required.

1.1 Research Objective

The primary objective is to build and evaluate a supervised multi-class classifier that predicts one of eight Plutchik-inspired emotion categories from text extracted from EmpatheticDialogues-style customer utterances. The model is intended for general text emotion analysis, not for clinical diagnosis or professional mental health assessment.

1.2 Target Labels

Index	Emotion Category
0	anger
1	anticipation
2	disgust
3	fear
4	joy
5	sadness
6	surprise
7	trust

2. Dataset Acquisition and Data Source

The dataset is obtained in Google Colab through KaggleHub. The notebook downloads the Kaggle dataset and selects the CSV file named emotion-emotion_69k.csv. In the recorded run, the original dataset contained 64,636 rows and 7 columns. The observed columns were Unnamed: 0, Situation, emotion, empathetic_dialogues, labels, Unnamed: 5, and Unnamed: 6.

```
Dataset source: atharvjairath/empathetic-dialogues-facebook-ai
Selected CSV: /kaggle/input/empathetic-dialogues-facebook-ai/emotion-emotion_69k.csv
Original shape: (64636, 7)
```

2.1 Raw Data Fields Used

Field	Role in Pipeline
Situation	Used to build a normalized situation_group for leakage-safe splitting.
emotion	Original emotion label before Plutchik mapping.
empathetic_dialogues	Dialogue text from which the customer-side text is extracted.
labels	Auxiliary dialogue text/label field present in the raw CSV; not the

	final target label.

3. Label Mapping to Plutchik Categories

The raw EmpatheticDialogues emotional labels are consolidated into eight Plutchik-inspired target classes. This mapping reduces the original emotional vocabulary into a fixed eight-class classification problem. Labels that cannot be mapped are removed to preserve target-label consistency.

3.1 Mapping Design

Plutchik Category	Mapped Source-Label Examples
joy	joyful, content, proud, excited
trust	trusting, faithful, caring, confident
fear	afraid, terrified, anxious, apprehensive
surprise	surprised, impressed
sadness	sad, lonely, devastated, disappointed, nostalgic, guilty, ashamed, sentimental
disgust	disgusted
anger	angry, furious, annoyed, jealous
anticipation	anticipating, hopeful, prepared

During cleaning, 41 unmapped records were removed. The notebook also removed 38 conflicting situation groups, where the same normalized situation group was associated with more than one target label. This step supports cleaner group-based splitting and reduces duplicate-leakage risks.

4. Text Preprocessing and Cleaning

The notebook performs text-focused cleaning before feature extraction. It extracts only the customer-side utterance from the dialogue string, strips whitespace, and discards rows with missing or empty input text. A normalized situation group is also created from the situation field by converting to lowercase and removing non-alphanumeric differences. This group field is used only for splitting and leakage checks, not as a direct model input.

4.1 Final Cleaned Dataset

Cleaning Output	Recorded Value
Removed unmapped labels	41
Conflicting situation groups removed	38
Final cleaned shape	(63,905, 4)
Number of final Plutchik labels	8

4.2 Label Distribution

Emotion	Samples	Share
anger	8,429	13.19%
anticipation	5,911	9.25%
disgust	2,010	3.15%
fear	7,693	12.04%
joy	10,523	16.47%
sadness	17,400	27.23%
surprise	5,251	8.22%
trust	6,688	10.47%

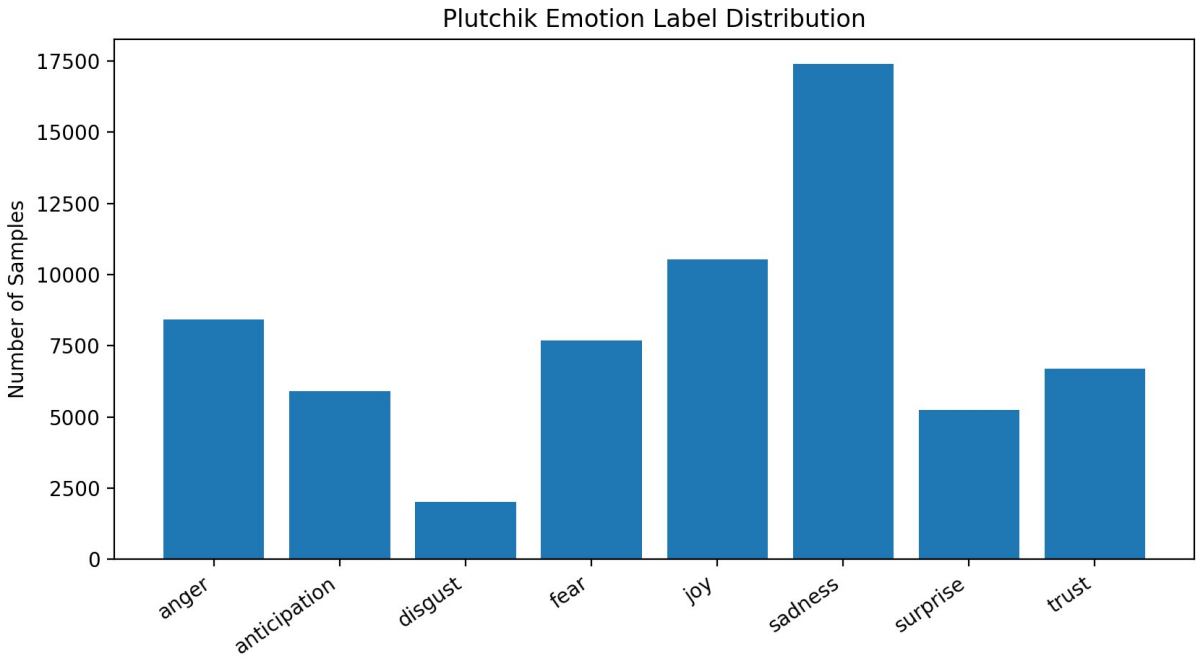

Figure 1. Plutchik emotion label distribution after cleaning and mapping.

The dataset is imbalanced. Sadness is the largest category, while disgust is the smallest. This imbalance motivated the evaluation emphasis on macro-averaged metrics and the use of class_weight=balanced in top-performing model configurations.

5. Train/Validation/Test Split Strategy

The pipeline uses a group-safe splitting strategy based on situation_group. This is important because EmpatheticDialogues contains multiple dialogue utterances that can share the same or highly similar situation. If such related situations appear across train, validation, and test sets, the classifier may appear stronger than it really is due to leakage. The recorded leakage checks reported zero overlap among train-validation, train-test, and validation-test group sets.

Split	Samples	Features Before Vectorization
Train	44,651	text, original_emotion, plutchik_label, situation_group
Validation	9,650	text, original_emotion, plutchik_label, situation_group
Test	9,604	text, original_emotion, plutchik_label, situation_group

5.1 Leakage Validation

Overlap Check	Result
Train-Validation overlap	0
Train-Test overlap	0
Validation-Test overlap	0

6. Feature Engineering

The classifier represents each text sample with a hybrid sparse feature vector built from word-level and character-level TF-IDF vectorizers. Word n-grams capture semantic and phrase-level cues, while character n-grams improve robustness to word variants, spelling differences, contractions, and short emotional expressions.

6.1 TF-IDF Representation

TF-IDF assigns higher weight to terms that occur often in a document but are less common across the corpus. This supports emotion classification because emotionally meaningful expressions can receive greater discriminative weight than very common words.

```
TF-IDF idea:
weight(term, document) = term_frequency(term, document) * inverse_document_frequency(term)
The notebook uses L2 normalization and sublinear term-frequency scaling.
```

6.2 Vectorizer Settings

Vectorizer	Analyzer	N-gram Range	Max Features	Other Settings
Word TF-IDF	word	(1, 3)	250,000	min_df=2, sublinear_tf=True, lowercase=True, norm=l2, strip_accents=unicode
Character TF-IDF	char_wb	(3, 5)	100,000	min_df=2, sublinear_tf=True, lowercase=True, norm=l2, strip_accents=unicode

6.3 Sparse Matrix Dimensions

Feature Matrix	Shape
Word TF-IDF train	(44,651, 250,000)
Character TF-IDF train	(44,651, 65,394)
Combined train	(44,651, 315,394)
Combined validation	(9,650, 315,394)
Combined test	(9,604, 315,394)

7. Model Training Procedure

The model family used in the pipeline is Linear Support Vector Classification. Linear SVMs are strong baselines for high-dimensional sparse text classification because the decision boundary can be learned efficiently in feature spaces created by TF-IDF n-grams.

7.1 Candidate Model Configurations

The notebook evaluates candidate LinearSVC configurations and ranks the strongest configurations using validation macro F1. Macro F1 is prioritized because it gives equal weight to every class, making it more informative under class imbalance than accuracy alone.

Rank	C	Class Weight	Validation Accuracy	Validation Macro Precision	Validation Macro Recall	Validation Macro F1	Validation Weighted F1
1	0.3	balanced	0.703938	0.687542	0.680162	0.682901	0.703259
2	0.5	balanced	0.703212	0.689313	0.677126	0.682167	0.702370
3	0.3	None	0.702694	0.703131	0.666008	0.681284	0.700150

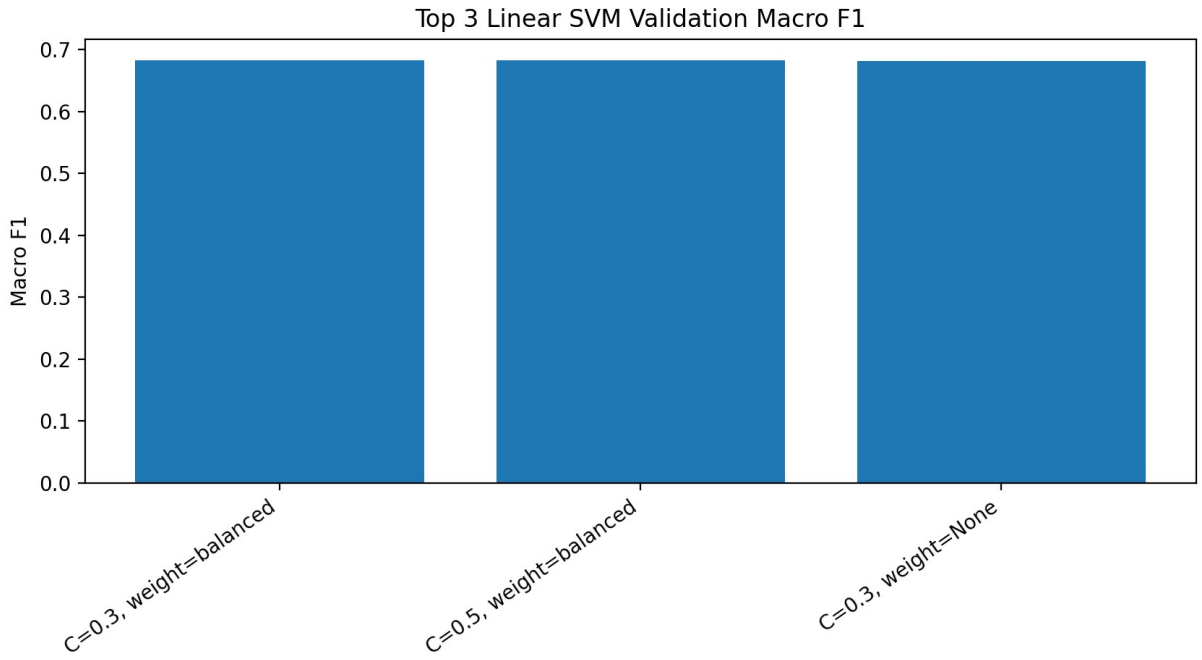

Figure 2. Validation macro F1 for the top three LinearSVC configurations.

7.2 Selected Model

Selected Parameter	Value
Classifier	LinearSVC
C	0.3
class_weight	balanced
Selection criterion	Highest validation macro F1
Validation accuracy	0.7039
Validation macro F1	0.6829
Validation weighted F1	0.7033

8. Evaluation Methodology

The final model is evaluated on a held-out test set that was not used for model fitting or hyperparameter selection. The evaluation includes accuracy, macro precision, macro recall, macro F1, weighted precision, weighted recall, weighted F1, per-class precision, per-class recall, per-class F1-score, class support, and raw confusion matrix analysis.

8.1 Metric Definitions

Metric	Interpretation
Accuracy	Overall proportion of correct predictions across all test samples.
Precision	For a class, the proportion of predicted samples for that class that are correct.
Recall	For a class, the proportion of true samples for that class that are correctly identified.
F1-score	Harmonic mean of precision and recall.
Macro average	Unweighted average across classes; useful for imbalanced data.
Weighted average	Average weighted by class support; influenced more by larger classes.
Support	Number of true samples per class in the test set.

9. Final Test Results

The selected LinearSVC model achieved approximately 70% accuracy on the held-out test set. Macro F1 was lower than weighted F1, indicating that smaller or more ambiguous categories were more difficult to classify than the majority categories.

Metric	Score
Accuracy	0.700021
Macro Precision	0.687799
Macro Recall	0.671641
Macro F1	0.678635
Weighted Precision	0.701321
Weighted Recall	0.700021
Weighted F1	0.700014

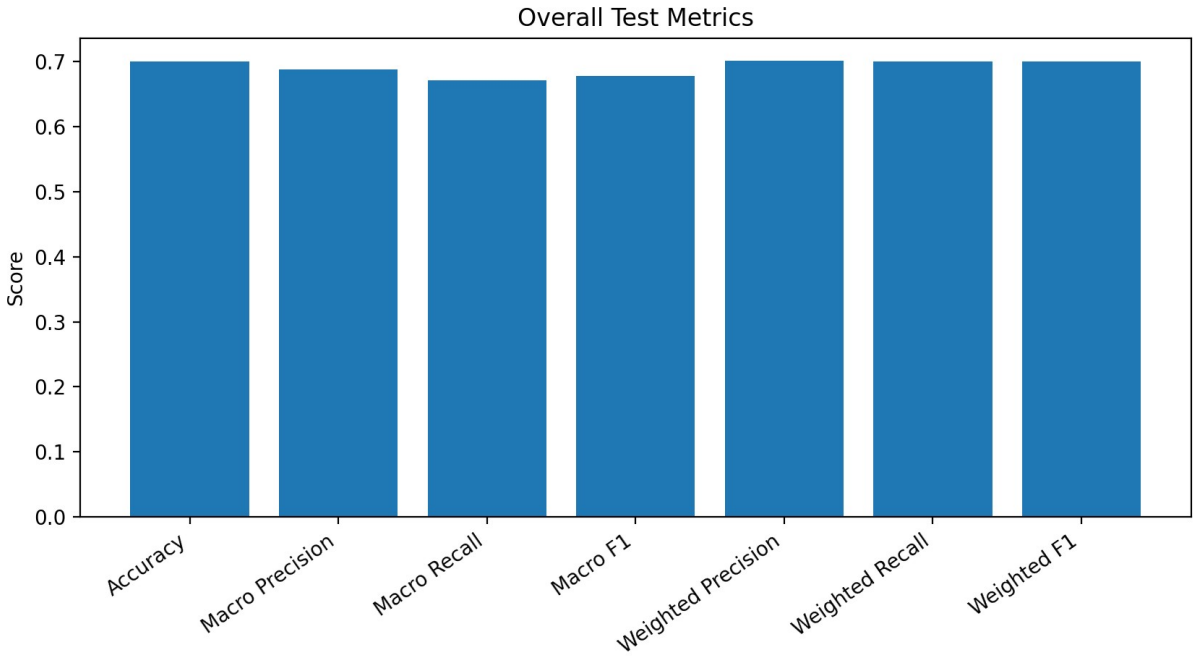

Figure 3. Overall held-out test metrics.

9.1 Per-Category Classification Report

Emotion	Precision	Recall	F1-score	Support
anger	0.6960	0.6932	0.6946	1,255
anticipation	0.5591	0.6175	0.5868	881
disgust	0.7462	0.6396	0.6888	308
fear	0.7770	0.7158	0.7451	1,168
joy	0.6748	0.7144	0.6940	1,586
sadness	0.7737	0.7929	0.7832	2,608
surprise	0.6508	0.6173	0.6336	797
trust	0.6249	0.5824	0.6029	1,001
Macro Avg	0.6878	0.6716	0.6786	9,604
Weighted Avg	0.7013	0.7000	0.7000	9,604

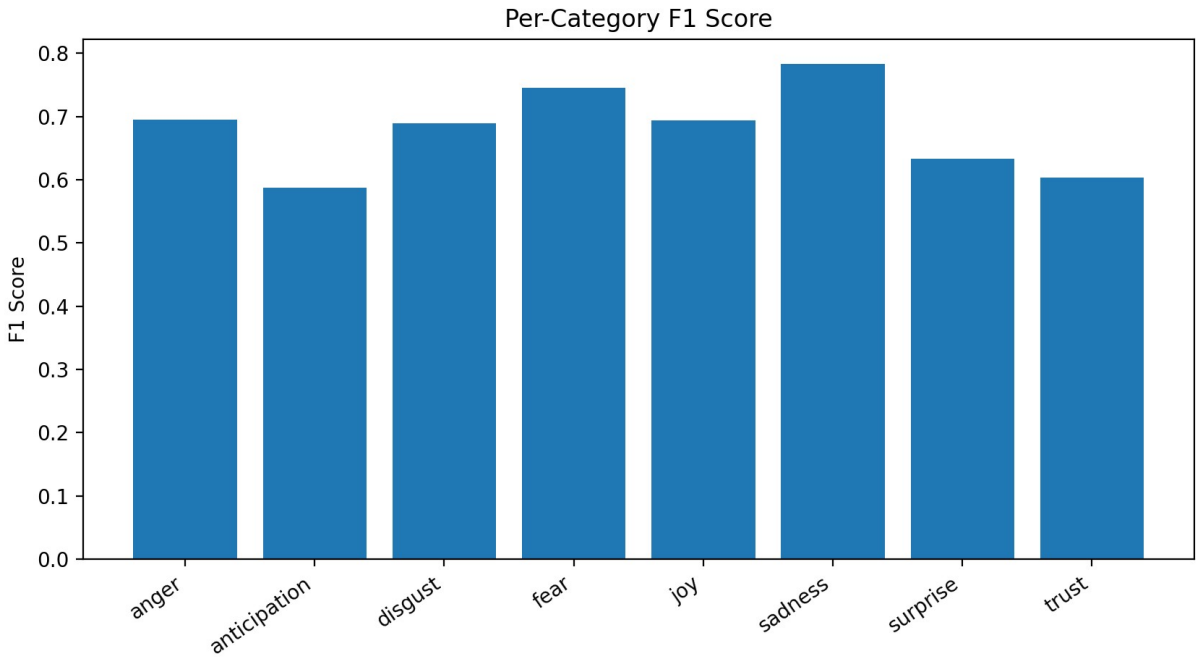

Figure 4. Per-category F1-score comparison.

9.2 Confusion Matrix Analysis

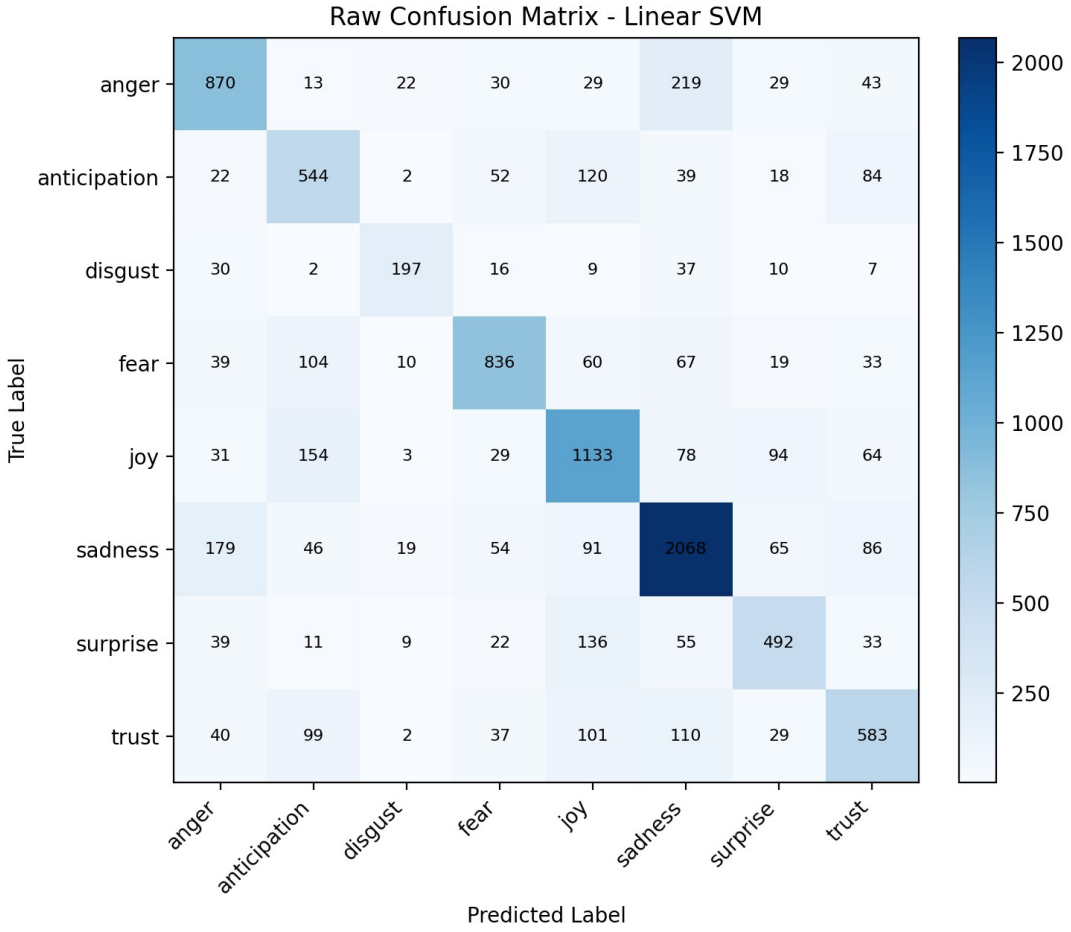

Figure 5. Raw confusion matrix for the final LinearSVC test predictions.

The confusion matrix shows that sadness was the strongest category in absolute correct predictions, with 2,068 true sadness samples classified correctly. Fear also performed strongly with 836 correct predictions out of 1,168 test samples. Anticipation and trust were comparatively more difficult, and several errors occurred among semantically related categories such as joy, anticipation, trust, sadness, and surprise. Anger was sometimes confused with sadness, suggesting lexical overlap between distress-related expressions.

10. Research Interpretation

10.1 Strengths of the Pipeline

- **Leakage-aware evaluation:** The train, validation, and test sets were separated by normalized situation group, reducing the risk of duplicated-context leakage.
- **Strong classical baseline:** The TF-IDF + LinearSVC design is efficient for high-dimensional sparse text features and provides interpretable reproducible baselines.
- **Balanced model selection:** Validation macro F1 was used for model selection, which is better aligned with imbalanced multi-class classification than accuracy alone.
- **Comprehensive reporting:** The notebook reports overall metrics, per-class precision/recall/F1/support, macro/weighted averages, and confusion matrix visualizations.

10.2 Key Findings

- The final model reached a test accuracy of 0.7000 and a weighted F1 of 0.7000, indicating stable overall classification performance.
- Macro F1 of 0.6786 indicates that minority or conceptually overlapping classes remain more difficult than majority classes.
- Sadness achieved the highest F1-score at 0.7832, supported by the largest number of training examples.
- Anticipation and trust achieved the lowest F1-scores, likely due to semantic overlap with joy, fear, and sadness in dialogue contexts.
- The use of class_weight=balanced slightly improved the selected model under the macro-F1 criterion.

10.3 Limitations

- **Heuristic label mapping:** The mapping from original EmpatheticDialogues labels to Plutchik categories is researcher-defined, so classification targets depend on the validity of that mapping.
- **Lexical feature limitation:** TF-IDF features do not model deep sentence context in the same way as transformer-based embeddings.
- **Class imbalance:** Sadness and joy have substantially more examples than disgust and surprise, which affects training dynamics and weighted metrics.
- **Dataset-specific behavior:** Results are specific to the selected Kaggle CSV and preprocessing choices. Different dataset revisions or mappings may produce different metrics.
- **Non-clinical scope:** The classifier predicts emotion categories from text and should not be interpreted as a diagnostic or mental health screening tool.

11. Saved Outputs and Reproducibility Package

The notebook saves the trained model, vectorizers, label encoder, and evaluation tables to allow reuse and GitHub-based reproducibility. The final outputs are compressed into a ZIP archive for download.

Artifact	Purpose
linear_svm_plutchik_model.pkl	Serialized best LinearSVC model.
word_vectorizer.pkl	Serialized word-level TF-IDF vectorizer.
char_vectorizer.pkl	Serialized character-level TF-IDF vectorizer.
label_encoder.pkl	Class-label encoder used to map text labels to integer IDs.

validation_results.csv	Validation metrics for candidate configurations.
classification_report.csv	Per-class precision, recall, F1-score, and support.
per_category_results.csv	Per-category summary metrics, including class-level accuracy/recall.
overall_test_metrics.csv	Final overall metrics.
confusion_matrix.csv	Raw confusion matrix.
normalized_confusion_matrix.csv	Row-normalized confusion matrix.
linear_svm_plutchik_outputs.zip	Compressed package for download and repository upload.

11.1 Recommended GitHub Repository Structure

```
empathetic-dialogues-tfidf-svm-emotion-classifier/
 README.md
 notebooks/
  empathetic_dialogues_tfidf_svm_emotion_classifier.ipynb
 outputs/
  classification_report.csv
  overall_test_metrics.csv
  confusion_matrix.csv
 models/
  linear_svm_plutchik_model.pkl
  word_vectorizer.pkl
  char_vectorizer.pkl
  label_encoder.pkl
 docs/
  training_process_documentation.pdf
 requirements.txt
 .gitignore
```

11.2 Reproduction Checklist

- Use the same Kaggle dataset source and selected CSV file.
- Set the same random seed used in the Colab notebook.
- Apply the same Plutchik label mapping dictionary.
- Remove unmapped labels and conflicting situation groups before splitting.
- Split by situation_group and verify zero group overlap across train, validation, and test.
- Fit TF-IDF vectorizers only on the training text, then transform validation and test text.
- Select the model using validation macro F1, not test performance.
- Evaluate the selected model once on the held-out test set.

12. End-to-End Algorithm Summary

1. Install and import required Python libraries in Google Colab.
2. Download the EmpatheticDialogues dataset through KaggleHub.
3. Locate and load emotion-emotion_69k.csv into a pandas DataFrame.
4. Map original emotion labels into eight Plutchik categories.
5. Remove unmapped labels and conflicting situation groups.
6. Extract customer-side dialogue text and construct final model input columns.
7. Create normalized situation_group values for leakage-safe splitting.
8. Perform train, validation, and test splitting by situation group.
9. Encode Plutchik labels into integer IDs using LabelEncoder.
10. Fit word-level and character-level TF-IDF vectorizers on training text only.
11. Transform train, validation, and test text into sparse TF-IDF matrices.
12. Horizontally stack word and character features into one combined matrix.
13. Train LinearSVC configurations and evaluate each on the validation set.
14. Select the best configuration using validation macro F1.

15. Evaluate the selected model on the held-out test set.
16. Generate overall metrics, classification report, per-category results, and confusion matrices.
17. Save the model, vectorizers, label encoder, CSV metrics, plots, and ZIP archive.

13. Recommendations for Future Improvement

- **Improve minority-class performance:** Experiment with targeted data augmentation, class-specific thresholds, or additional source-label review for disgust, anticipation, surprise, and trust.
- **Validate label mapping:** Ask domain reviewers to inspect the Plutchik mapping because mapping quality directly controls target quality.
- **Add error analysis:** Export high-confidence wrong predictions and examine recurring lexical patterns responsible for confusion.
- **Compare non-transformer baselines:** Benchmark Logistic Regression, Complement Naive Bayes, LinearSVC, and calibrated SVM variants using the same split.
- **Add transformer comparison only as optional baseline:** If compute allows, compare against RoBERTa/BERT to quantify the performance gap between lexical and contextual methods.
- **Version every artifact:** Save dataset hash, random seed, library versions, and commit ID in the repository to make results auditable.

14. Ethical and Practical Considerations

Emotion classification systems can support user-facing reflection, journaling analytics, and affective computing research, but they must be presented with appropriate limitations. The model infers likely text emotion categories and can be wrong, especially for sarcasm, mixed emotions, coded language, multilingual input, or emotionally ambiguous statements. For any mental health-related application, results should be framed as supportive signals rather than clinical conclusions.

References

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273-297.

Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features. European Conference on Machine Learning.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Plutchik, R. (1980). A general psychoevolutionary theory of emotion. In R. Plutchik & H. Kellerman (Eds.), Emotion: Theory, research, and experience: Vol. 1. Theories of emotion. Academic Press.

Rashkin, H., Smith, E. M., Li, M., & Boureau, Y.-L. (2019). Towards empathetic open-domain conversation models: A new benchmark and dataset. Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523.